

Вопросы государственной экзаменационной комиссии

и подробные ответы по докладу и презентации

*ВКР «Разработка веб-приложения для организации личного файлового хранилища»
(LocalCloud)*

180 вопросов с подробными ответами · материал для подготовки к защите

1. Актуальность и предметная область

1. В чём актуальность вашей работы?

Ответ. Объём личных данных растёт, а основной способ их хранения — публичные облака с подписочной моделью: это постоянные растущие платежи и потеря контроля над данными (провайдер может менять тарифы, блокировать аккаунт, прекратить обслуживание; после 2022 года добавились трудности с оплатой). Альтернатива — самостоятельное размещение, но существующие решения требуют мощных серверов. Актуальна задача создать лёгкое хранилище, работающее на бюджетном сервере 1 CPU / 1 ГБ ОЗУ.

2. Какова основная инженерная проблема, которую вы решаете?

Ответ. Передача и обработка объёмов данных, многократно превышающих оперативную память сервера. Файл в десятки гигабайт не должен проходить через память приложения, упаковка папки в архив не должна требовать памяти под весь архив, а генерация превью не должна конкурировать с обработкой запросов за процессор.

3. Что такое self-hosted и почему он востребован?

Ответ. Self-hosted (самостоятельное размещение) — развёртывание ПО на собственном или арендованном сервере под полным контролем владельца. Востребован по экономическим (фиксированная аренда вместо растущей подписки), организационным (независимость от провайдера) и правовым причинам (локализация персональных данных, 152-ФЗ).

4. Кто целевая аудитория продукта?

Ответ. Частные пользователи и небольшие команды (ориентир 3–5 активных пользователей), которым нужно личное файловое хранилище с приватностью и контролем над данными при минимальных затратах на инфраструктуру.

5. Какие нормативные документы регулируют предметную область?

Ответ. Прежде всего Федеральный закон 152-ФЗ «О персональных данных» с требованиями к обработке и локализации; для зарубежных пользователей — GDPR. Размещение данных на своём сервере в выбранной юрисдикции упрощает соблюдение требований локализации.

6. Почему вы считаете, что 1 ядро и 1 ГБ памяти — реалистичное ограничение?

Ответ. Это типовая конфигурация самого доступного по цене VPS, который реально может позволить себе частный пользователь. Если решение работает на ней, оно работает и на более мощных серверах; конфигурация масштабируется параметрами `.env`.

7. Объект и предмет исследования?

Ответ. Объект — программные средства организации личного файлового хранилища на основе веб-технологий в условиях самостоятельного размещения. Предмет — архитектурные решения и методы хранения, передачи и разграничения доступа к файлам при ограниченных ресурсах сервера (presigned-передача, multipart-загрузка, фоновые задачи).

8. Какие сценарии деятельности пользователей вы анализировали?

Ответ. Загрузка и хранение фото-, видео- и документальных архивов; организация в папки; скачивание (в т.ч. наборов файлов архивом); предоставление доступа другим людям и публикация по ссылке; удаление с возможностью восстановления.

9. Чем личное хранилище отличается от корпоративного?

Ответ. Личное рассчитано на одного владельца и малое число доверенных пользователей, минимальную инфраструктуру и простоту развёртывания; корпоративное — на масштабирование, отказоустойчивость, интеграцию с корпоративными системами и сложные политики, что избыточно для целевого профиля.

2. Цель, задачи и требования

10. Сформулируйте цель работы.

Ответ. Разработать веб-приложение LocalCloud, организующее личное файловое хранилище с загрузкой/скачиванием больших файлов, иерархией данных, разграничением доступа и публикацией по ссылкам, сохраняющее работоспособность на сервере 1 CPU / 1 ГБ ОЗУ.

11. Какие задачи решались для достижения цели?

Ответ. Пять задач: анализ области и формирование требований; проектирование архитектуры (С4), БД, API и подсистем; реализация серверной и клиентской частей и средств развёртывания; тестирование (модульное, интеграционное, нагрузочное, безопасности); технико-экономическое обоснование и охрана труда.

12. Как вы собирали и документировали требования?

Ответ. По рекомендациям стандарта IEEE 830-1998: на основе анализа прецедентов выделены акторы, построена диаграмма вариантов использования, сформированы функциональные требования (сводная таблица) и нефункциональные требования по категориям.

13. Перечислите ключевые функциональные требования.

Ответ. Управление файлами и папками, многочастичная загрузка, скачивание архивами, корзина, общий доступ между пользователями и публичные ссылки, квоты, превью, аутентификация и регистрация по заявке, администрирование.

14. Перечислите нефункциональные требования.

Ответ. Производительность (1 CPU/1 ГБ, размер файла не ограничен памятью), безопасность (JWT, хэши паролей, проверка прав), надёжность (отсутствие потери данных, повтор задач, миграции), удобство (адаптивный интерфейс, темы), развёртывание и сопровождение (одна команда, конфигурация под хост).

15. Как проверялось выполнение нефункциональных требований?

Ответ. Производительность и поведение при перегрузке — нагрузочным тестированием ($p99 \leq 177$ мс при требовании 2 с); безопасность — модульными и системными тестами; надёжность — проверкой восстановления незавершённых загрузок и повторов задач.

16. Что выходит за границы продукта (ограничения)?

Ответ. Сознательно исключены десктоп- и мобильные клиенты-синхронизаторы (доступ только через браузер), версионирование файлов и сквозное шифрование на стороне клиента — это упрощает систему под целевой профиль и ресурсы.

17. Какие допущения вы приняли при проектировании?

Ответ. Целевой профиль 3–5 пользователей и единицы фоновых задач в минуту; единственный сервер (без кластера); доверенная локальная сеть между сервисами в Docker; администрирование выполняет владелец.

18. Как требования трассируются до реализации?

Ответ. Каждый прецедент главы 1 отражён в доменном сервисе, эндпоинте API и тесте; каждой сущности концептуальной модели соответствует таблица БД (таблица соответствия), что обеспечивает прослеживаемость от анализа к коду.

19. Чем подтверждается достижение цели?

Ответ. Готовый к развёртыванию продукт работает на целевой конфигурации; файловый трафик идёт в обход сервера, что снимает зависимость от размера файла; качество подтверждено 7679 тестами и покрытием 98,6 %/90,2 %.

3. Анализ аналогов и обоснование разработки

20. Какие аналоги вы рассматривали?

Ответ. Пять: публичные облака Google Drive, Яндекс Диск, Dropbox и самостоятельно размещаемые Nextcloud и Seafile. Сравнение проведено по критериям стоимости, приватности, требований к ресурсам и функциональности.

21. В чём недостаток существующих self-hosted решений?

Ответ. Nextcloud, ownCloud и Seafile пропускают файловый трафик через серверное приложение, поэтому загрузка/скачивание больших файлов создаёт высокую нагрузку на CPU и память и требует мощных серверов, недоступных рядовому пользователю.

22. Какую свободную нишу вы выявили?

Ответ. Лёгкое самостоятельно размещаемое хранилище для сервера минимальной конфигурации, где файловый трафик не проходит через приложение, а ресурсоёмкие операции вынесены в фон.

23. Чем ваш продукт лучше Nextcloud?

Ответ. Он рассчитан на 1 CPU/1 ГБ за счёт presigned-передачи в обход backend, разворачивается одной командой с автоподбором конфигурации и не тянет тяжёлую платформу плагинов; при этом покрывает базовые сценарии личного хранилища.

24. Почему не доработать готовый Nextcloud вместо своей разработки?

Ответ. Архитектурно Nextcloud проксирует файлы через PHP-приложение; устранить это означало бы переписать ядро. Для целевого ограничения по памяти

потребовалась иная архитектура с самого начала, поэтому разработка собственного решения оправдана.

25. Почему не использовать просто FTP/SFTP или сетевую папку?

Ответ. Они не дают веб-доступа, разграничения прав на уровне приложения, публичных ссылок с паролем и сроком, корзины, квот, превью и аудита — то есть пользовательских сценариев облачного хранилища.

4. Архитектура системы

26. Какой архитектурный стиль вы выбрали и почему?

Ответ. Многоуровневая (слоистая) клиент-серверная архитектура с выделенным объектным хранилищем и отдельным фоновым обработчиком. Она проще микросервисной, соответствует масштабу задачи и обеспечивает чёткое разделение ответственности.

27. Что такое модель C4 и зачем вы её применяли?

Ответ. C4 — способ описания архитектуры на четырёх уровнях детализации: контекст, контейнеры, компоненты, код. Я применял её, чтобы наглядно показать систему от окружения до внутренней структуры; в работе обоснована её применимость и к немикросервисным системам.

28. Из каких контейнеров состоит система (уровень C2)?

Ответ. Из семи единиц: одностраничного приложения в браузере и шести серверных контейнеров — шлюза nginx, API (FastAPI), фонового worker'a, PostgreSQL и MinIO; всё управляется Docker Compose.

29. Почему именно 7 «контейнеров», но 6 сервисов?

Ответ. В терминах C4 контейнер — это исполняемая единица; SPA выполняется в браузере пользователя и считается отдельным контейнером, а на сервере работают шесть docker-сервисов. Отсюда 7 контейнеров и 6 сервисов.

30. Опишите ключевой маршрут данных.

Ответ. Файловый трафик идёт от браузера через шлюз напрямую в MinIO по подписанным ссылкам, минуя API. API лишь подписывает ссылки и хранит метаданные. Поэтому потребление памяти/CPU сервера не зависит от размера файла.

31. Какова роль шлюза nginx?

Ответ. Единая точка входа: маршрут / отдаёт статику SPA, /api проксируется в API, пути бакетов — в MinIO. Единый origin избавляет от CORS и скрывает внутренние сервисы; шлюз также восстанавливает внутренний Host для корректной подписи presigned-ссылок.

32. Перечислите слои серверного приложения.

Ответ. Маршрутизаторы (api) → доменные сервисы (services) → репозитории (repositories) поверх Unit of Work → модели (models); доступ к хранилищу инкапсулирован фасадом storage. Зависимости направлены строго вниз.

33. Как обеспечивается отсутствие циклических зависимостей?

Ответ. Каждый слой знает только о нижележащем: маршрутизаторы — о сервисах, сервисы — о Unit of Work и фасаде хранилища, репозитории — о моделях. Это подтверждается диаграммой пакетов, где обратных рёбер нет.

34. Зачем выделен отдельный фоновый worker, а не фоновые задачи внутри API?

Ответ. Чтобы ресурсоёмкие операции (архивы, превью) не конкурировали с обработкой пользовательских запросов за CPU и память и имели независимые лимиты ресурсов. Worker — отдельный процесс с собственными ограничениями, переиспользующий доменный код.

35. Как взаимодействуют API и worker?

Ответ. Через таблицу задач в PostgreSQL: API ставит задачу, worker опрашивает очередь, захватывает задачи блокировкой и исполняет обработчики. Общего кода (доменных сервисов) они используют один и тот же, но развёртываются отдельно.

36. Почему вы не выбрали микросервисную архитектуру?

Ответ. Микросервисы оправданы при больших командах и нагрузках; для одного исполнителя и сервера минимальной конфигурации они дали бы накладные расходы на сеть, инфраструктуру и эксплуатацию без выигрыша. Слоистый монолит + отдельный worker проще и эффективнее.

37. Где хранятся метаданные и где содержимое файлов?

Ответ. Метаданные (узлы, права, квоты, задачи) — в PostgreSQL; содержимое файлов, превью и архивы — в объектном хранилище MinIO. Разделение зафиксировано уже на концептуальном уровне модели данных.

38. Что показывает диаграмма развёртывания?

Ответ. Физическое размещение: на одном сервере в Docker работают шлюз, API, worker, PostgreSQL и MinIO; браузер пользователя обращается к шлюзу, а за файлами — напрямую к MinIO через шлюз.

39. Масштабируется ли архитектура горизонтально?

Ответ. Да, частично: API и worker — без состояния и допускают несколько экземпляров (worker уже безопасно работает в нескольких копиях за счёт SKIP LOCKED); PostgreSQL и MinIO потребуют отдельных решений кластеризации, что выходит за целевой профиль.

5. База данных

40. Сколько таблиц в базе данных и каковы её контуры?

Ответ. Четырнадцать таблиц, сгруппированных в три контура: файловой иерархии (узлы, файлы, папки), доступа (права, публичные ссылки, сессии) и служебный (загрузки и их части, квоты, задачи, аудит, заявки на регистрацию, токены, корзина).

41. Как вы моделируете файлы и папки?

Ответ. Через таблицу узла `file_system_nodes` со ссылкой на родителя (список смежности); специфичные атрибуты вынесены в таблицы-спутники `files` и `folders`, связанные с узлом «один к одному». Общие операции (перемещение, права, корзина) работают единообразно с узлом.

42. Почему первичные ключи — UUID, а не автоинкремент?

Ответ. UUID генерируются на стороне приложения (не нужен round-trip к БД), безопасны для показа в URL (не раскрывают количество и порядок объектов) и удобны при распределённой работе.

43. До какой нормальной формы нормализована схема?

Ответ. До третьей нормальной формы; сознательные денормализации (поля `path` и `depth` на узле) добавлены для ускорения построения хлебных крошек и не нарушают целостность, так как поддерживаются приложением.

44. Как реализовано мягкое удаление?

Ответ. Признаком `is_deleted` и моментом удаления на узле плюс таблицей-журналом `trash_items` (кто, когда удалил, состояние элемента). Частичный индекс по (`parent_id`, `name`) построен только для активных узлов.

45. Зачем частичный индекс по активным узлам?

Ответ. Он одновременно ускоряет листинг папки и гарантирует уникальность имени среди активных детей, не учитывая узлы в корзине, — благодаря чему можно переиспользовать имя удалённого файла.

46. Как выполняются обходы поддеревьев (размер папки, каскад, права)?

Ответ. Рекурсивными CTE-запросами: один запрос поднимается/спускается по ссылке `parent_id`, что заменяет обход по одному узлу (антипаттерн N+1) и работает эффективно на глубоких деревьях.

47. Как настроены каскадные правила внешних ключей?

Ответ. Удаление пользователя каскадно удаляет его узлы; ссылки на исполнителя действия (`granted_by`, `created_by`) при удалении субъекта обнуляются правилом SET NULL, что сохраняет историю действий.

48. Почему полезная нагрузка задач хранится в jsonb?

Ответ. Структура нагрузки зависит от типа задачи, и жёсткая схема была бы избыточной; `jsonb` даёт гибкость и при этом индексируем и запрашиваем средствами PostgreSQL.

49. Как эволюционирует схема базы данных?

Ответ. Через миграции Alembic (их семь), которые применяются автоматически при старте контейнера API и идиempотентны. Это позволяет безопасно обновлять схему без потери данных.

50. Почему PostgreSQL, а не MySQL или NoSQL?

Ответ. Нужны надёжные транзакции, внешние ключи, рекурсивные CTE, частичные индексы, типы `enum`, `jsonb` и блокировки строк с SKIP LOCKED для очереди задач — всё это есть в PostgreSQL. NoSQL не дал бы транзакционной целостности метаданных.

51. Как обеспечивается целостность между метаданными и содержимым?

Ответ. Фоновая задача проверки целостности периодически сверяет записи БД с объектами в MinIO и выявляет «осиротевшие» данные; при загрузке объект подтверждается до фиксации метаданных.

52. Что хранит таблица refresh-токенов?

Ответ. Цепочку ротации: каждый токен ссылается на родительский и заменивший его, что позволяет обнаружить повторное использование уже отозванного токена (признак компрометации) и отозвать всю цепочку.

53. Как считается занятый объём для квот?

Ответ. Узлы и размеры файлов агрегируются; квота хранит отдельные счётчики (объём, число файлов, ссылок, активных загрузок), а периодическая фоновая задача пересчитывает их для устранения возможных расхождений.

6. REST API

54. В каком стиле спроектирован API?

Ответ. В стиле REST: ресурсы именуются существительными, операции выражаются методами HTTP, состояние между запросами не хранится (сессия — в подписанных cookie). Все маршруты версионированы префиксом `/api/v1`.

55. Сколько эндпоинтов и групп ресурсов в API?

Ответ. Около 104 эндпоинтов в 15 группах ресурсов (auth, registration, users, quotas, nodes, folders, uploads, downloads, trash, permissions, public-links, tasks, audit, config, health).

56. Как документируется контракт API?

Ответ. Спецификацией OpenAPI 3.1, которая генерируется непосредственно из кода средствами FastAPI, поэтому контракт и реализация не могут разойтись; в debug-режиме доступны Swagger UI и ReDoc.

57. Как обрабатываются ошибки?

Ответ. Единым форматом: любой неуспешный ответ возвращает структуру с кодом HTTP, машиночитаемым типом, человекочитаемым сообщением и идентификатором запроса для трассировки. Нарушение схемы даёт код 422 с указанием поля.

58. Зачем версионировать API префиксом?

Ответ. Чтобы развивать контракт, не ломая существующих клиентов: несовместимые изменения вводятся новым префиксом (`/api/v2`), а старая версия некоторое время сосуществует.

59. Как реализована страничная навигация?

Ответ. Списочные эндпоинты принимают limit и offset и возвращают элементы вместе с метаданными пагинации (общее число, лимит, смещение), что позволяет клиенту строить навигацию.

60. Как разграничен доступ к маршрутам?

Ответ. Тремя категориями на уровне зависимостей FastAPI: публичные (вход, заявка, доступ по токenu), пользовательские (требуют аутентификации) и административные (требуют роль admin); для узлов дополнительно проверяется право на узел.

61. Чем валидируются входные данные?

Ответ. Схемами Pydantic v2: типы, форматы, ограничения проверяются до попадания в бизнес-логику; при нарушении возвращается 422 с детализацией поля.

62. Возвращает ли API файловый поток?

Ответ. Как правило нет: для скачивания и загрузки выдаются presigned-ссылки на MinIO, и трафик идёт мимо API. Небольшие операции (например, отдача метаданных, превью-ссылок) обслуживаются обычными JSON-ответами.

63. Как клиент узнаёт о включённых возможностях?

Ответ. Через публичный эндпоинт /config, который отдаёт флаги функциональности (превью, просмотрщик, проигрывание, редактирование); это позволяет отключать тяжёлые функции на слабом сервере без пересборки клиента.

7. Хранение файлов, MinIO, multipart и presigned-ссылки

64. Что такое presigned URL и зачем он нужен?

Ответ. Это временная подписанная ссылка, дающая клиенту прямой доступ к объекту в хранилище на ограниченное время и операцию. Она позволяет браузеру загружать/скачивать файлы напрямую из MinIO, не пропуская трафик через сервер приложения.

65. Что такое многочастичная (multipart) загрузка?

Ответ. Загрузка файла по частям: клиент инициирует сессию, получает presigned-ссылки на части, заливает их напрямую в MinIO и подтверждает; затем сервер завершает сборку объекта. Размер части — 8 МБ (минимум для последней части у S3 — 5 МБ).

66. Зачем нужна именно многочастичная загрузка?

Ответ. Она позволяет грузить файлы, превышающие память, поддерживает докачку и параллелизм частей, а также даёт прогресс. Сервер не буферизует файл — он лишь координирует сессию и счётчики.

67. Что делает сервер во время загрузки, если файл идёт мимо него?

Ответ. Создаёт upload-сессию и записи частей, выдаёт presigned-ссылки, принимает подтверждения частей с их ETag, проверяет квоту и ёмкость, а по завершении фиксирует метаданные файла. Сами байты через него не проходят.

68. Почему MinIO, а не хранение в файловой системе или в БД?

Ответ. MinIO даёт S3-совместимый протокол с presigned-ссылками (ключевое для обхода сервера), управление объектами и масштабируемость. Хранение в БД раздуло бы её и нагрузило память; «голая» ФС не даёт presigned-доступа и удобного объектного API.

69. Что такое S3-совместимость и зачем она?

Ответ. S3 — де-факто стандарт объектного хранилища; совместимость означает единый API. Благодаря ей MinIO при необходимости заменяется на облачный S3 без изменения кода приложения.

70. Как решается проблема подписи presigned-ссылок за шлюзом?

Ответ. Backend подписывает ссылки под внутренним именем хоста MinIO; шлюз nginx при проксировании восстанавливает этот Host, иначе подпись S3 оказалась бы недействительной (ошибка SignatureDoesNotMatch).

71. Как формируются ключи объектов в хранилище?

Ответ. По детерминированной схеме на основе идентификаторов пользователя и файла (и отдельного пространства для превью). Ключ хранится в метаданных файла и связывает запись БД с объектом.

72. Как работает скачивание папки или набора файлов?

Ответ. Постановкой фоновой задачи: worker потоково упаковывает выбранные объекты в ZIP в MinIO, не держа архив целиком в памяти, после чего клиенту выдаётся временная ссылка на готовый архив.

73. Как генерируются превью?

Ответ. Фоновым worker'ом: изображения — через Pillow, PDF — через PyMuPDF, видео — через ffmpeg. Лимиты и качество настраиваются, а на слабом сервере генерацию превью можно полностью отключить флагом.

74. Что происходит при отмене или сбое загрузки?

Ответ. Незавершённые сессии истекают по сроку; фоновая задача очистки удаляет просроченные сессии и неподтверждённые части (abort multipart в MinIO), освобождая место и счётчики квоты.

75. Как проверяется, что хватает места под загрузку?

Ответ. Перед началом проверяются квота пользователя и ёмкость сервера (с учётом доли диска, по умолчанию 85 %). Для согласованного учёта при конкурентных загрузках используется advisory-блокировка и кэш ёмкости.

76. Как обеспечивается целостность загруженного файла?

Ответ. Каждая часть подтверждается своим ETag, итоговая сборка проверяется хранилищем; опционально вычисляется контрольная сумма. Фоновая сверка целостности дополнительно контролирует соответствие БД и объектов.

77. Можно ли заменить MinIO на облачный S3?

Ответ. Да: благодаря S3-совместимости достаточно изменить параметры подключения и публичный хост в .env; код приложения не меняется.

78. Является ли единственный узел MinIO точкой отказа?

Ответ. Да, в целевой конфигурации MinIO однонодовый — это единая точка отказа диска, поэтому критичен регулярный бэкап. При необходимости MinIO разворачивается в распределённом режиме, но это выходит за целевой профиль.

79. Как устроен бэкап содержимого и метаданных?

Ответ. Двумя согласованными операциями в одно окно обслуживания: логический дамп PostgreSQL (pg_dump) и снимок данных MinIO (tar тома). По отдельности они бесполезны, поэтому делаются вместе и хранятся вне сервера.

8. Фоновый worker и очередь задач

80. Как реализована очередь фоновых задач?

Ответ. Таблицей задач в PostgreSQL: задачи имеют тип, статус, полезную нагрузку (jsonb), число попыток и время следующего запуска. Worker опрашивает очередь пачками и исполняет обработчики из реестра по типу задачи.

81. Почему очередь в БД, а не Redis или RabbitMQ?

Ответ. При целевых масштабах (единицы задач в минуту) транзакционная очередь с SELECT FOR UPDATE SKIP LOCKED даёт те же гарантии доставки и

атомарности, что и брокер, но не требует ещё одного сервиса в памяти сервера минимальной конфигурации.

82. Как несколько воркеров не берут одну задачу дважды?

Ответ. Запросом `SELECT ... FOR UPDATE SKIP LOCKED`: он блокирует выбранные строки и пропускает уже занятые другими процессами, поэтому каждая задача достаётся ровно одному воркеру без распределённого координатора.

83. Что произойдёт, если воркер упадёт во время выполнения задачи?

Ответ. Задача захватывается с ограниченным временем владения (поля `locked_by/locked_until`); по истечении срока «зависшая» задача снова становится доступной и будет подхвачена другим воркером.

84. Как обрабатываются повторные попытки при ошибке?

Ответ. Неудачная задача повторяется с экспоненциально растущей задержкой до предела числа попыток; это сглаживает временные сбои инфраструктуры и не перегружает систему.

85. Какие периодические задачи выполняет планировщик?

Ответ. Очистку корзины, истёкших загрузок и публичных ссылок, пересчёт квот и сверку целостности базы данных с хранилищем.

86. Как обеспечивается идемпотентность задач?

Ответ. Через ключ идемпотентности и проверку статуса: повторное исполнение той же задачи не приводит к дублированию результата (например, повторная сборка архива не создаёт второй объект).

87. Как ограничивается параллелизм в воркере?

Ответ. Семафором: одновременно выполняется ограниченное число обработчиков, чтобы ресурсоёмкие операции (превью, архивы) не исчерпали CPU и память сервера.

88. Почему worker переиспользует код API?

Ответ. Чтобы бизнес-правила (права, квоты, работа с хранилищем) были едиными и не дублировались; `worker` и `API` используют один доменный слой, но запускаются как разные процессы с разными лимитами.

89. Как пользователь узнаёт о прогрессе фоновой задачи?

Ответ. Через эндпоинты задач: клиент опрашивает статус и прогресс по идентификатору задачи и по завершении получает результат (например, ссылку на готовый архив).

9. Безопасность и разграничение доступа

90. Как устроена аутентификация?

Ответ. На JWT-токенах: при входе выдаётся пара токенов (access и refresh) в httpOnly-cookie. Access короткоживущий, refresh ротируется. Cookie недоступны JavaScript, что защищает от кражи токенов через XSS.

91. Почему токены в httpOnly-cookie, а не в localStorage?

Ответ. localStorage доступен скриптам страницы, поэтому уязвим к XSS-краже токена. httpOnly-cookie скриптам недоступен; в сочетании с единым origin и SameSite это закрывает и XSS-кражу, и CSRF.

92. Что такое ротация refresh-токена и зачем она?

Ответ. При каждом обновлении выдаётся новый refresh-токен, а старый отзывается. Если отозванный токен используется повторно, это признак компрометации — вся цепочка отзывается. Это ограничивает ущерб от утечки токена.

93. Как защищаетесь от CSRF?

Ответ. Фронтенд и API живут на одном origin (через шлюз), а cookie помечены SameSite. Это исключает межсайтовую отправку запросов; отдельный CSRF-токен при едином origin не требуется.

94. Как хранятся пароли?

Ответ. Только в виде стойких хэшей (bcrypt/argon2 через passlib) с солью; исходные пароли нигде не сохраняются и не логируются. Проверка — сравнением хэшей.

95. Опишите модель прав доступа.

Ответ. Пять уровней на узел: чтение, скачивание, запись, удаление и владелец. Право выдаётся пользователю на конкретный узел и наследуется на всё его поддерево; эффективное право вычисляется с учётом наследования.

96. Как реализовано наследование прав без потери производительности?

Ответ. Одним рекурсивным CTE: запрос поднимается по цепочке предков узла (пропуская удалённых) и сразу джойнится с таблицей прав. Это заменяет обход предков по одному (N+1) и работает на глубоких деревьях.

97. Как защищены публичные ссылки?

Ответ. Ссылка имеет тип (просмотр/скачивание/загрузка), срок действия и необязательный пароль (хранится хэшем). После нескольких неудачных попыток пароля проверки временно блокируются, что защищает от перебора.

98. Как защищаетесь от перебора паролей и токенов?

Ответ. Ограничением частоты запросов: для чувствительных auth-точек — порядка 10 запросов с одного IP за 5 минут, для проверки публичных ссылок — около 30 за 5 минут; для пароля ссылки — блокировка после серии неудач.

99. Как предотвращаются SQL-инъекции?

Ответ. Все запросы строятся через SQLAlchemy с параметризацией; пользовательский ввод не конкатенируется в SQL, поэтому инъекции исключены.

100. Как предотвращается XSS?

Ответ. Хранением токенов в httpOnly-cookie, экранированием вывода во фронтенде (React по умолчанию экранирует) и политикой безопасности контента; пользовательские имена файлов трактуются как данные, а не как разметка.

101. Что проверяется при каждой операции над узлом?

Ответ. Аутентификация пользователя и наличие требуемого права на конкретный узел (с учётом наследования): чтение — для просмотра, запись — для изменения, удаление — для перемещения в корзину и т.д.

102. Как изолируются данные разных пользователей?

Ответ. Каждый узел принадлежит владельцу, и доступ другим возможен только через явно выданное право; запросы всегда фильтруются по пользователю и проверке прав, поэтому увидеть чужие файлы без выданного доступа нельзя.

103. Что такое регистрация по заявке и зачем она?

Ответ. Новый пользователь подаёт заявку, которую одобряет администратор; до одобрения учётная запись неактивна. Это предотвращает бесконтрольную саморегистрацию на личном сервере.

104. Ведётся ли журнал действий?

Ответ. Да, журнал аудита фиксирует действия пользователей и системные события (вход, операции над узлами, административные действия), что важно для разбора инцидентов и подотчётности.

105. Какие меры защиты секретов предусмотрены?

Ответ. Секреты (ключ подписи, пароли БД/MinIO/админа) задаются через `.env` и не попадают в репозиторий; вне `debug`-режима приложение на старте отказывается подниматься с дефолтными/`placeholder`-значениями, а генератор создаёт случайные секреты.

106. Что проверяет тестирование безопасности?

Ответ. Подсистему JWT (срок, подпись, издатель, аудитория, типы токенов), хэширование и политику паролей, проверки прав и их наследование, защиту от перебора и гигиену секретов — на модульном, интеграционном и системном уровнях.

107. Поддерживается ли HTTPS?

Ответ. Да: для продакшена приложен пример конфигурации `nginx` с TLS 1.2/1.3, HSTS и редиректом HTTP→HTTPS; при этом включается флаг безопасных `cookie` (Secure).

10. Технологический стек и обоснование выбора

108. Почему серверная часть на Python и FastAPI?

Ответ. Асинхронная модель `FastAPI`/`uvicorn` позволяет одному процессу на одном ядре обслуживать множество одновременных запросов, ожидающих ввода-вывода (БД, хранилище). Это критично для сервера минимальной конфигурации.

109. Почему SQLAlchemy 2 и asyncpg?

Ответ. `SQLAlchemy 2` даёт типизированный декларативный ORM и мощный построитель запросов (включая рекурсивные CTE), а `asyncpg` — быстрый асинхронный драйвер PostgreSQL. Вместе они дают производительный неблокирующий доступ к данным.

110. Почему клиент на React и TypeScript?

Ответ. `React` — зрелая экосистема (маршрутизация, состояние серверных данных, виртуализация списков), а статическая типизация `TypeScript` сокращает дефекты интеграции с API. `Vite` обеспечивает быструю сборку и горячую перезагрузку.

111. Зачем TanStack Query и TanStack Virtual?

Ответ. Query управляет кэшированием и синхронизацией серверного состояния (загрузки, инвалидация), а Virtual виртуализирует длинные списки файлов, отрисовывая только видимые элементы — это держит интерфейс отзывчивым на больших папках.

112. Почему Tailwind CSS и Radix UI?

Ответ. Tailwind даёт консистентный стиль без тяжёлого UI-фреймворка, а примитивы Radix (через shadcn) обеспечивают доступные и предсказуемые компоненты (диалоги, меню), которые легко стилизовать.

113. Зачем Docker и Docker Compose?

Ответ. Контейнеризация даёт воспроизводимое окружение и изоляцию сервисов, а Compose поднимает весь стек из шести сервисов одной командой с лимитами ресурсов и healthcheck'ами — это и есть «развёртывание одной командой».

114. Что такое uv и зачем он?

Ответ. uv — быстрый менеджер зависимостей и окружений Python; он детерминированно устанавливает пакеты по lock-файлу, ускоряя сборку образа и локальную разработку.

115. Почему Pydantic v2?

Ответ. Он даёт строгую валидацию и сериализацию данных на границе API с высокой производительностью (ядро на Rust) и тесно интегрирован с FastAPI для автогенерации OpenAPI.

116. Почему выбраны именно эти версии (Python 3.13, React 19, PostgreSQL 16)?

Ответ. Использованы актуальные стабильные версии на момент разработки, дающие современные возможности языка и библиотек и длительную поддержку; конкретные версии зафиксированы в lock-файлах для воспроизводимости.

117. Чем обеспечивается единый стиль кода?

Ответ. Линтером и форматтером: для Python — ruff, для фронтенда — ESLint и Prettier; единые правила проверяются автоматически и поддерживают читаемость кода.

118. Не избыточен ли такой стек для личного хранилища?

Ответ. Нет: каждый компонент решает конкретную задачу целевого ограничения по ресурсам (асинхронность, presigned-передача, фоновые задачи, кэширование на клиенте). Стек подобран минимально достаточным, а тяжёлые функции отключаемы.

119. Как конфигурация подстраивается под ресурсы сервера?

Ответ. Скрипт-генератор определяет ресурсы хоста, берёт их долю (по умолчанию 85 %) и рассчитывает параметры .env (воркеры, пул соединений, лимиты памяти, флаги превью), а также генерирует случайные секреты.

11. Реализация, паттерны и качество кода

120. Какие шаблоны проектирования вы применили?

Ответ. Repository, Unit of Work, Facade (StorageService), Adapter (асинхронная обёртка MinIO SDK), Dependency Injection (механизм FastAPI), Command + Registry (обработчики фоновых задач) и Proху (ленивый прокси настроек).

121. Как работает Unit of Work в вашей реализации?

Ответ. Это транзакционный контейнер, владеющий асинхронной сессией БД и фабрикой репозиториев; изменения фиксируются только явным commit, а выход из контекста без фиксации автоматически откатывает транзакцию, исключая частичную запись.

122. Зачем нужен фасад StorageService?

Ответ. Он предоставляет единый высокоуровневый интерфейс к бакетам, объектам, multipart-операциям и подписи ссылок, скрывая детали MinIO SDK от доменных сервисов и упрощая замену хранилища.

123. Где применяется Adapter?

Ответ. В асинхронной обёртке блокирующего MinIO SDK поверх пула потоков: синхронные вызовы SDK выполняются в отдельных потоках, чтобы не блокировать событийный цикл asyncio.

124. Как реализован Dependency Injection?

Ответ. Через систему зависимостей FastAPI: сервисы и текущий пользователь внедряются в обработчики как зависимости, что развязывает слои и упрощает подмену реализаций заглушками в тестах.

125. Зачем связка Command + Registry для задач?

Ответ. Каждый тип задачи имеет обработчик, зарегистрированный в реестре по типу; диспетчер исполняет их единообразно, не зная деталей, а добавление нового типа задачи не требует изменения диспетчера.

126. Что даёт паттерн Proxu для настроек?

Ответ. Ленивый прокси откладывает чтение конфигурации до первого обращения, что исключает побочные эффекты при импорте модулей и упрощает тестирование с разными конфигурациями.

127. Каков объём исходного кода?

Ответ. Около 126 тысяч строк исходного текста серверной и клиентской частей без учёта тестов; это косвенно подтверждает оценку трудоёмкости методом PERT.

128. Как обеспечивается читаемость и сопровождаемость кода?

Ответ. Чёткой слоистой структурой, единым стилем (ruff/ESLint/Prettier), документированием модулей и функций, типизацией (Python type hints, TypeScript strict) и высоким покрытием тестами.

12. Тестирование и обеспечение качества

129. Сколько тестов в проекте и какие уровни?

Ответ. Всего 7679 автоматических тестов: 6745 серверных (6559 модульных и 186 интеграционных) и 934 клиентских. Уровни — модульный, интеграционный и системный.

130. Какое покрытие кода достигнуто?

Ответ. Серверная часть — 98,6 % операторов (из 22 086 не покрыто 308), клиентская — 90,2 % по операторам, 91,9 % по строкам, 86,0 % по функциям и 84,2 % по ветвям при закреплённых порогах 85 % и 80 %.

131. Какими инструментами выполняется тестирование?

Ответ. Серверная часть — pytest (с покрытием через coverage.py), клиентская — Vitest с Testing Library; нарушение порогов покрытия приводит к падению сборки.

132. Как тестируется API на интеграционном уровне?

Ответ. Через TestClient к собранному приложению FastAPI с подменой внешних зависимостей заглушками: проверяются маршрутизация, валидация схем, коды ответов, формат ошибок и заголовки безопасности без реальных БД и MinIO.

133. Почему интеграционные тесты не требуют внешних сервисов?

Ответ. Репозитории и фасад хранилища подменяются заглушками, поэтому прогон детерминирован и быстр; реальная инфраструктура нужна только для системных и нагрузочных тестов.

134. Как проводилось нагрузочное тестирование?

Ответ. На развёрнутом стенде через шлюз по производственному пути инструментом на `httpx/asyncio`; измерялись задержки при штатной нагрузке и поведение при перегрузке.

135. Каковы результаты нагрузочного тестирования?

Ответ. При штатной нагрузке задержка p99 не превышала 177 мс при требовании 2 с; при перегрузке поведение оставалось управляемым (быстрый отказ с кодом 503 вместо деградации).

136. Какое узкое место вы выявили и как его устранять?

Ответ. При высокой конкурентности потребление соединений пула БД: аутентификационная зависимость и доменный сервис берут по соединению, что при ~120 одновременных запросах истощает пул. Краткосрочно — ограничитель не выше половины ёмкости пула; долгосрочно — переиспользование одной сессии на запрос.

137. Что такое p99 и почему именно он?

Ответ. p99 — 99-й перцентиль распределения задержки: 99 % запросов обслуживаются быстрее этого значения. Он показывает «почти худший» случай и важнее среднего, так как отражает опыт самых медленных запросов.

138. Проводилось ли тестирование совместимости?

Ответ. Да, кросс-браузерная проверка интерфейса в основных современных браузерах; SPA использует стандартные веб-API без зависимости от конкретного движка.

139. Как оформлены результаты тестирования?

Ответ. Системные тест-кейсы описаны идентификатором, сценарием, ожидаемым результатом и статусом; сводные количественные результаты и динамика дефектов приведены таблицами и сводным дашбордом.

140. Как фиксировались и устранялись дефекты?

Ответ. Дефекты документировались и исправлялись коммитами с префиксом fix;; характерные примеры сведены в таблицу. Высокое покрытие тестами предотвращает регрессии при изменениях.

13. Развёртывание и эксплуатация

141. Как развёртывается система?

Ответ. Одной командой `docker compose up`: поднимаются шесть сервисов, API при первом старте сам применяет миграции, создаёт администратора и бакеты; конфигурация берётся из `.env`, подобранного генератором под ресурсы хоста.

142. Какие сервисы поднимаются и какие порты открыты наружу?

Ответ. `nginx`, `frontend`, `API`, `worker`, `PostgreSQL` и `MinIO`. Наружу открыт только шлюз (порт 80/443); `PostgreSQL` и `MinIO` не публикуются на хост, консоль `MinIO` доступна лишь локально.

143. Как обновлять систему без потери данных?

Ответ. `git pull` и пересборка образов (`docker compose up -d --build`); данные хранятся в томах `PostgreSQL` и `MinIO` и сохраняются. После пересборки API шлюз перезапускается для сброса кэша адресов.

144. Что делать при сбое одного из сервисов?

Ответ. Перезапуск любого сервиса не приводит к потере данных: незавершённые загрузки и задачи либо истекают, либо возобновляются. Состояние — только в томах БД и `MinIO`.

145. Как организован мониторинг работоспособности?

Ответ. Эндпоинтами `health` (`live/ready` и детальная диагностика БД и хранилища) и `healthcheck`'ами в `Compose`; логи доступны через `docker compose logs`.

146. Какие требования к окружению для эксплуатации?

Ответ. `Docker` с плагином `Compose`; сервер минимальной конфигурации 1 CPU / 1 ГБ для целевого профиля; для публичного доступа — домен и TLS-сертификаты.

147. Как выполняется резервное копирование в эксплуатации?

Ответ. Две команды на `cron` хоста: логический дамп `PostgreSQL` и `tar`-снимок тома `MinIO`, выполняемые в одно окно; копии хранятся вне сервера. Восстановление — в чистые тома.

148. Какая документация подготовлена для сопровождения?

Ответ. README монорепозитария (быстрый старт, бэкап/восстановление, типовые неисправности), справочники серверной и клиентской частей, документация REST API, руководство по подбору .env и CHANGELOG.

14. Процесс разработки и контроль версий

149. Какую методологию разработки вы применяли?

Ответ. Подход, близкий к Kanban: непрерывный поток небольших задач без церемоний спринтов, избыточных для одного исполнителя; каждая задача — отдельная ветка, завершающаяся слиянием.

150. Какую стратегию ветвления вы использовали?

Ответ. Упрощённый Git Flow: постоянные ветки main (стабильные выпуски) и develop (интеграция), короткоживущие ветки feature/*, release/* и hotfix/*; после слияния ветки задач удаляются.

151. Сколько коммитов и как они оформлены?

Ответ. Свыше 80 коммитов по конвенции Conventional Commits (тип, область, краткое описание в повелительном наклонении), что делает историю самодокументируемой и позволяет собирать перечень изменений.

152. Как организовано версионирование продукта?

Ответ. По семантическому версионированию (SemVer) MAJOR.MINOR.PATCH; первый функционально полный выпуск помечен тегом v0.1.0, история изменений ведётся в CHANGELOG по соглашению Keep a Changelog.

153. Где размещён исходный код?

Ответ. В публичном репозитории на GitHub по адресу github.com/magomedov-dev/LocalCloud, с README, лицензией, документацией и историей коммитов.

154. Использовали ли вы автоматизацию проверок?

Ответ. Да: тесты, линтеры и форматтеры запускаются для контроля качества; пороги покрытия закреплены в конфигурации и нарушение приводит к падению сборки.

15. Техничко-экономическое обоснование

155. Как оценивалась трудоёмкость разработки?

Ответ. Методом экспертных оценок PERT по пяти этапам жизненного цикла (анализ и проектирование, кодирование, тестирование, документация, развёртывание): по трём оценкам (оптимистичной, вероятной, пессимистичной) рассчитывалась ожидаемая трудоёмкость.

156. Почему именно PERT?

Ответ. Он применим при индивидуальной разработке без накопленной статистики аналогов и учитывает неопределённость через три оценки. Результат косвенно подтверждается фактическим объёмом кода (~126 тыс. строк).

157. Какова себестоимость разработки?

Ответ. Полная себестоимость по статьям калькуляции составила 802 141 рубль; договорная цена с НДС — 1 203 211 рублей.

158. Как оценивался экономический эффект для тиражируемого продукта?

Ответ. Как совокупная экономия владельцев развёртываний относительно подписочных облаков: годовая экономия одного развёртывания — 38 400 рублей по сравнению с платным тарифом сопоставимого объёма.

159. Каковы итоговые показатели эффективности?

Ответ. При консервативном прогнозе тиражирования чистый дисконтированный доход за пять лет — 5 817 859 рублей, индекс доходности — 8,25, дисконтированный срок окупаемости — 1,6 года.

160. Что входит в статьи себестоимости?

Ответ. Оплата труда разработчика и руководителя (с часовыми ставками от окладов), страховые взносы 30,2 %, амортизация оборудования, электроэнергия и накладные расходы.

161. Какие качественные эффекты у проекта?

Ответ. Повышение приватности и контроля над персональными данными, снижение зависимости от внешних провайдеров, вклад в экосистему открытого ПО и возможность дальнейшего развития продукта.

162. Почему продукт окупается так быстро?

Ответ. Это тиражируемое ПО с открытым кодом: разовые затраты на разработку распределяются на множество развёртываний, каждое из которых ежегодно экономит владельцу подписочные платежи, поэтому совокупная экономия быстро превышает затраты.

16. Безопасность жизнедеятельности и охрана труда

163. Какие вредные и опасные факторы вы рассмотрели?

Ответ. Для рабочего места программиста: недостаточную/неравномерную освещённость, неблагоприятный микроклимат, зрительное и нервно-психическое напряжение, шум техники и опасность поражения электрическим током.

164. Как организовано рабочее место?

Ответ. По ГОСТ 12.2.032-78 и санитарным нормам: площадь не менее 4,5 м² и объём не менее 15 м³ на место, высота рабочей поверхности 725 мм, регулируемое подъёмно-поворотное кресло, продуманное размещение оборудования.

165. Какие требования к микроклимату?

Ответ. Работа относится к категории Ia (лёгкая, сидя); оптимальные параметры температуры, влажности и скорости воздуха установлены по СанПиН 1.2.3685-21 и поддерживаются отоплением и вентиляцией.

166. Как обеспечивается электро- и пожарная безопасность?

Ответ. Помещение относится к категории без повышенной опасности поражения током (ПУЭ) и к категории В4 по пожарной опасности (СП 12.13130.2009), класс возможного пожара — Е; предусмотрены соответствующие средства защиты.

167. Как организован режим труда и отдыха?

Ответ. Регламентированными перерывами 10–15 минут при работе с ПК через установленные интервалы, что снижает зрительное и нервно-психическое напряжение.

168. Рассматривались ли вопросы экологичности?

Ответ. Да: продукт — ПО, его эксплуатация энергоэффективна (работа на сервере минимальной конфигурации снижает энергопотребление по сравнению с мощными серверами), предусмотрена корректная утилизация оборудования по окончании срока службы.

17. Общие, перспективные и каверзные вопросы

169. В чём научная и практическая новизна работы?

Ответ. Практическая новизна — набор архитектурных решений (presigned multipart в обход сервера, очередь задач в реляционной СУБД, наследуемая модель прав на

дерево), обеспечивающих работу полноценного файлового хранилища на сервере минимальной конфигурации; эти решения переиспользуемы и за пределами задачи.

170. Что было самым сложным в разработке?

Ответ. Согласовать прямую presigned-передачу файлов с проверкой прав, квот и целостности на стороне сервера, не пропуская при этом файловый трафик через приложение, а также корректно организовать конкурентную очередь задач на одной БД.

171. Что бы вы улучшили или сделали иначе?

Ответ. Устранил бы выявленное узкое место с пулом соединений (переиспользование сессии на запрос), добавил бы антивирусную проверку загружаемых файлов и опциональное шифрование на стороне сервера, а также десктоп-клиент синхронизации.

172. Каковы перспективы развития продукта?

Ответ. Клиенты синхронизации, сквозное шифрование, полнотекстовый поиск по содержимому, совместное редактирование, распределённый режим MinIO для отказоустойчивости и интеграция с внешними провайдерами идентификации.

173. Как система поведёт себя при росте числа пользователей?

Ответ. До десятков пользователей — за счёт конфигурации (воркеры, пул, лимиты); далее потребуется вертикальное масштабирование сервера и устранение узкого места пула, а при больших нагрузках — кластеризация PostgreSQL и MinIO, что предусмотрено архитектурой.

174. Что произойдёт, если два пользователя одновременно создадут файл с одинаковым именем в одной папке?

Ответ. Уникальность имени среди активных детей гарантируется частичным индексом по (parent_id, name); вторая операция получит конфликт (409), и клиенту будет предложено переименование.

175. Где в системе возможна потеря данных и как вы её предотвращаете?

Ответ. Потенциально — при сбое между записью объекта и фиксацией метаданных или при удалении тома. Предотвращается транзакциями, подтверждением объекта до фиксации, сверкой целостности и обязательным согласованным бэкапом обоих хранилищ.

176. Почему вы уверены в корректности прав доступа?

Ответ. Права проверяются централизованно на каждом обращении к узлу, наследование вычисляется одним детерминированным запросом, а подсистема прав покрыта модульными и интеграционными тестами, включая граничные случаи наследования.

177. Чем ваш подход к загрузке принципиально отличается от обычной загрузки через сервер?

Ответ. При обычной загрузке файл проходит через память/диск сервера, и его размер ограничен ресурсами. У нас браузер пишет части напрямую в хранилище по подписанным ссылкам, поэтому сервер не тратит память на файл и его размер ограничен только квотой и диском.

178. Как обеспечивается отзывчивость интерфейса на больших папках?

Ответ. Виртуализацией списков (TanStack Virtual): отрисовываются только видимые элементы, миниатюры подгружаются лениво и кэшируются, а серверное состояние кэшируется TanStack Query.

179. Соответствует ли реализация спроектированной архитектуре?

Ответ. Да: серверный код разделён по слоям в точном соответствии с диаграммой компонентов, каждой сущности модели соответствует таблица, а контракт API совпадает с реализацией, так как OpenAPI генерируется из кода.

180. Какой главный вывод по результатам работы?

Ответ. Поставленная цель достигнута: создан готовый к развёртыванию продукт, который благодаря передаче файлов в обход сервера обслуживает файлы, многократно превышающие его память, при подтверждённом тестами качестве и экономической эффективности.